

The Knowledge Graph Playbook for Agentic Systems: Anthropic Engineering Guide

A Synthesis for Study

Based on Anthropic's Knowledge Graph Cookbook, Building Effective AI Agents, Managed Agents, and Claude API docs

Independently compiled, July 2026 — not affiliated with or endorsed by Anthropic

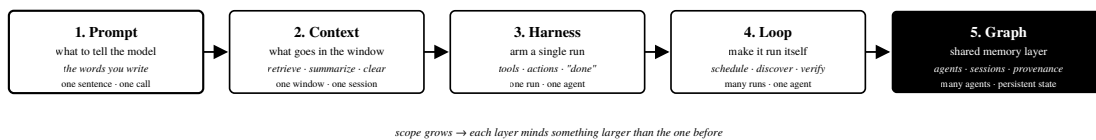


Fig. 1. The five-layer stack, horizontal. Each layer removes the practitioner from one more position of manual work. Graph engineering (black) is the shared memory layer — the world model that agents read from, write to, and reason over across sessions.

Abstract—Graph Engineering is the discipline of designing, building, and maintaining the structured memory layer that multi-agent systems need but context windows cannot provide. While prompt engineering teaches what to say, context engineering teaches what to load, and loop engineering teaches how to schedule — graph engineering teaches how to give agents a shared world model that outlives any single conversation. This note defines the discipline, identifies the three problems that retrieval-augmented generation cannot solve (multi-hop reasoning, cross-agent state sharing, and provenance-carrying fact-checking), and shows how Anthropic's published tools — structured outputs, the Model Context Protocol, and composable agent patterns — provide the building blocks for production knowledge graphs. We ground every claim in Anthropic's public cookbook, engineering blog, and API documentation, and present a concrete architecture for wiring a knowledge graph into each of the five documented agent patterns. The central argument: as generation becomes cheap and agents multiply, the bottleneck shifts from producing answers to knowing which facts to trust. The knowledge graph is the infrastructure that makes trust traceable.

Index Terms—Graph engineering, knowledge graphs, multi-agent systems, structured outputs, Model Context Protocol, Claude, agentic AI, provenance, fact-checking.

I. GRAPH ENGINEERING AS A DISCIPLINE

The "XX Engineering" progression has a clear direction. Prompt engineering minds what to tell the model. Context engineering minds what goes into the window. Loop engineering minds how to make it run itself. Each layer removes the practitioner from one more position of manual work. Graph engineering is the next layer: it minds the shared memory that agents read from, write to, and reason over — the structured world model that no single context window can hold.

The need is concrete. When Anthropic's own engineering team scaled internal agent deployments, they encountered what they call the context-management problem: "context grows too complex for one agent to manage effectively, creating performance bottlenecks as agents struggle to maintain coherence." The problem is worse in multi-agent systems. Five workers, each with findings, each with evidence, must

coordinate without passing complete transcripts through an orchestrator whose window is already full. The knowledge graph is the architectural answer: a typed, provenance-carrying, queryable structure that lives outside every agent's window and survives every session flush.

This is not a theoretical concern. Anthropic published a complete knowledge-graph cookbook showing how to build one entirely from Claude API calls — no trained NER models, no relation classifiers, no entity-resolution heuristics. The Pydantic schema is the only "training data." The cookbook demonstrates the pipeline on an Apollo-program corpus and reports precision/recall against a hand-labeled gold set. What the cookbook does not do, and what this note adds, is map the pipeline onto the full space of agent architectures and explain where, specifically, the graph changes what agents can do.

II. THREE PROBLEMS RAG CANNOT SOLVE

Retrieval-augmented generation retrieves text chunks by semantic similarity and feeds them into the context window. This works for single-hop questions where the answer exists in one passage. Three classes of problems defeat it.

A. Multi-Hop Reasoning

The question "which locations are connected to people who flew on Apollo 11" requires chaining person → mission edges from one document with person → location edges from another. The two passages share no lexical overlap with each other or with the question. RAG cannot retrieve what it cannot match. A knowledge graph makes the connection explicit: the person node has edges to both the mission and the

location, and graph traversal discovers the chain regardless of surface similarity. In the Apollo cookbook, the grounded answer cites specific edges — "(Armstrong) --[walked on]--> (Moon)" — while the ungrounded answer draws on pretraining and produces plausible but untraceable claims. On a private corpus, only the grounded answer works at all.

B. Cross-Agent State Sharing

When an orchestrator delegates to five workers, each produces findings in its own context. The orchestrator must somehow combine them. The RAG approach — each worker retrieves its own chunks — gives no shared state. Worker A's findings are invisible to Worker B. The knowledge graph solves this structurally: workers write entities and relations to a shared graph. Worker B can query for anything Worker A found. The orchestrator's window stays small because the shared state lives outside it. This is the multi-agent analogue of Anthropic's Managed Agents principle: "the session is not the context window." The graph is the session.

C. Provenance-Carrying Fact-Checking

An evaluator agent needs to check whether a generator's claim is correct. With RAG, it can retrieve a passage that might contain supporting evidence — but the passage is free-form text, and "does this passage support this claim" is itself a judgment call that can hallucinate. With a knowledge graph, the evaluator queries for the specific triple the generator asserts and gets a binary answer: the triple exists (with predicate P, from document D) or it does not. The feedback shifts from "this seems off" to "triple (X, works_at, Y) does not exist; the graph contains (X, left, Y) from

document Z." That specificity is what makes an evaluator-optimizer loop converge rather than drift.

III. THE BUILDING BLOCKS

Anthropic's platform provides three building blocks that make knowledge-graph construction practical without external NLP infrastructure.

A. Structured Outputs

Structured outputs guarantee that Claude's response validates against a Pydantic schema and returns as a typed Python object. This eliminates the parsing, validation, and coercion failures that plague free-form extraction at scale. The extraction schema for a knowledge graph is compact: an `Entity` with name, type, and description; a `Relation` with source, predicate, and target; and an `ExtractedGraph` containing lists of both. The schema is the interface contract between the extraction stage and the resolution stage — and it is the only "training" the extractor needs. Change the schema and you change what the extractor finds, with no retraining, no labeled data, and no domain-specific model.

B. The Model Context Protocol (MCP)

MCP is the open standard that connects agents to external systems — databases, APIs, issue trackers, and knowledge stores. For graph engineering, MCP serves two roles. First, it is how the extraction pipeline connects to source documents: an MCP connector reads from a document store, and the extractor processes what it receives. Second, and more importantly, it is how agents query and update the graph at runtime: an MCP-based knowledge-graph server exposes the graph as a tool that any agent can call, reading subgraphs and writing new triples without the

agent needing to know the storage backend. Anthropic open-sourced a Knowledge Graph Memory MCP Server that stores entities and relations in a local JSON file — a minimal but functional implementation that demonstrates the pattern. The pattern itself scales to Neo4j, Neptune, or Postgres.

C. The Two-Model Strategy

The cookbook uses Haiku for extraction and Sonnet for resolution and querying. This is not arbitrary — it follows from the cost and capability profile of each stage. Extraction is high-volume and schema-constrained: Haiku is fast, cheap, and sufficient because the structured-output schema constrains the output space. Resolution requires weighing conflicting evidence across documents ("Edwin Aldrin" vs. "Buzz Aldrin" — same person, zero string overlap): Sonnet's stronger reasoning handles this. Querying requires multi-hop reasoning over serialized triples: again, Sonnet. The two-model strategy is itself a form of routing — the cheapest model that handles the subtask is the right one, which is the same principle Anthropic's agent guidance recommends for all agentic architectures.

IV. WIRING THE GRAPH INTO AGENT PATTERNS

Anthropic documents five composable agent patterns. Each has a specific integration point with the knowledge graph.

TABLE I
WHERE THE GRAPH FITS IN EACH PATTERN

Pattern	Without graph	With graph
<i>Augmented LLM</i>	Retrieves text chunks by similarity	Traverses typed edges for multi-hop answers with citations
<i>Prompt chaining</i>	Passes output to next stage as text	Writes entities to graph between stages; next stage queries graph

Pattern	Without graph	With graph
<i>Routing</i>	Classifies input with an LLM call	Looks up entity type and degree in graph — no LLM call needed
<i>Orchestrator-workers</i>	Workers pass summaries through orchestrator's window	Workers read/write shared graph; orchestrator window stays clean
<i>Evaluator-optimizer</i>	Evaluator reads output and guesses correctness	Evaluator queries graph for specific triples and cites provenance

The most impactful integration is with the evaluator-optimizer pattern, because it changes the *quality* of the evaluator's signal. Without the graph, the evaluator's verdict is an opinion — "this looks wrong." With the graph, the verdict is a fact-check — "this triple does not exist in the graph, but this related triple does." The difference is the difference between an evaluator that hallucinates approval and one that catches real errors. This is why Anthropic's cookbook emphasizes provenance: every edge carries its source document so that the evaluator can trace any claim to its origin.

V. WHAT THE PIPELINE REPLACES

The classical knowledge-graph pipeline requires three separately trained models: a Named Entity Recognition model (trained on domain-labeled spans), a Relation Extraction model (trained on labeled entity pairs), and an Entity Resolution system (hand-tuned string-similarity heuristics with blocking rules). Each requires labeled data. Each breaks when moved to a new domain. Each must be maintained as the data shifts. The Claude-based pipeline (Fig. 1) replaces all three with structured-output prompts. The adaptation cost drops from weeks of labeling and training to hours of prompt tuning.

The trade-off is precision at the margins. A trained NER model can learn fine-grained entity distinctions (distinguishing a gene name from a protein name in biomedical text) that a prompted extractor handles less reliably without domain-specific prompt engineering. But for the vast majority of enterprise use cases — where the entity types are PERSON, ORGANIZATION, LOCATION, EVENT, PRODUCT — the prompted approach matches or exceeds trained-model quality at a fraction of the deployment cost. The cookbook reports perfect precision (1.00) on its test corpus, meaning everything Haiku extracted was correct. Recall is lower (0.38–0.55), but this is controlled by the extraction prompt and can be tuned through the evaluation feedback loop.

VI. THE ENTITY RESOLUTION PROBLEM

Entity resolution is where the knowledge graph is won or lost. Raw extraction produces a fractured graph where the same real-world entity appears as multiple disconnected nodes: "NASA" and "National Aeronautics and Space Administration," "Neil Armstrong" and "Neil Alden Armstrong," "Buzz Aldrin" and "Edwin Aldrin." Without resolution, multi-hop reasoning breaks — the path from Armstrong to the Moon passes through two different Armstrong nodes that don't connect.

String similarity handles typos and capitalization variants. It fails completely on nickname resolution ("Edwin Aldrin" → "Buzz Aldrin" — zero character overlap), abbreviation expansion ("NASA" → full name), and cross-lingual variants. Claude-based resolution handles all three because it uses the one-line descriptions from extraction as semantic context. The description "Aldrin — second person to walk on

the Moon, Lunar Module Pilot of Apollo 11" and "Edwin Aldrin — American astronaut, second man on the Moon" contain enough semantic overlap for Sonnet to cluster them, even though the names share no characters.

Two failure modes matter in production. Under-resolution leaves duplicates in the graph, fragmenting paths and reducing recall. Over-resolution merges distinct entities (folding "Gemini 12" into "Project Gemini"), creating false connections and reducing precision. The evaluation harness catches both: precision drops when over-resolution creates false edges, recall drops when under-resolution fragments real paths. The feedback loop — change the resolution prompt, rerun the scorer, watch the metrics move — is what makes the pipeline self-improving.

VII. FROM EXTRACTION TO WORLD MODEL

A graph of labels is not yet a world model. The step that transforms one into the other is entity summarization: pooling every mention of a hub entity across all source documents, adding its graph neighborhood as context, and synthesizing a structured profile with key facts, time ranges, and traceable claims. The Apollo-program node, for example, draws on all six source documents to produce a profile spanning its 1960 conception, 1969 lunar landing, and 1973 end — information no single document contained in full.

The world model is what downstream agents actually use. A research agent queries it for background on an entity before writing a report. An evaluator agent checks claims against its key facts. A planning agent reads the graph neighborhood to understand what entities are connected to its current task. The richer the profiles, the more useful the graph becomes as a

reasoning substrate — but richness costs Sonnet calls, so summarization should be applied selectively to high-degree nodes that tie the most documents together.

VIII. GRAPH ENGINEERING IN PRODUCTION

A. Incremental Construction

A production graph is never built once. New documents arrive daily; the graph must absorb them without rebuilding. The correct pattern is incremental: extract entities from the new document, resolve them against the existing canonical set (not against each other), add new edges, and re-summarize only the entities whose source-document set changed. The state file records which documents have been processed. The graph accumulates rather than rebuilds — this is the knowledge-graph equivalent of "the agent forgets, the repo does not."

B. Cost Management

Extraction (Haiku) is cheap and scales linearly with corpus size — prompt caching and the Message Batches API (50% discount for 24-hour latency tolerance) make it economical even for large corpora. Resolution (Sonnet) is moderate: one call per entity type per resolution pass. Summarization (Sonnet with large context) is expensive per entity but sublinear in corpus size because it applies only to high-degree nodes. Querying (Sonnet) is per-question. The dominant cost for a large corpus is extraction; the dominant cost for a heavily-queried graph is querying. Optimizing both independently is why the two-model strategy matters.

C. Storage Backend

NetworkX handles graphs up to a few hundred thousand edges. Beyond that: Neo4j or Neptune for property-graph queries, or three Postgres tables (`entities`, `relations`, `aliases`) with recursive CTEs for traversal. The extraction and resolution code does not change — only the persistence layer. For teams starting today, the MCP Knowledge Graph Memory Server (a JSON file) is the right first step: simple, local, inspectable. Graduate to a database when query volume or graph size demands it.

D. Monitoring

Four signals matter: extraction rate (entities per document — sudden drops signal corpus shift), resolution compression ratio (raw names / canonical entities — near 1.0 means consistent naming, above 2.0 means heavy variation), graph connectivity (number of connected components — growth means resolution is missing links), and query latency (time from question to grounded answer). Each signal has a corresponding failure mode, and each failure mode has a specific fix.

IX. THE EVALUATION FEEDBACK LOOP

The evaluation harness is what turns the pipeline from a demo into a production system. The loop has three steps: change the extraction prompt, rerun the scorer against a gold set, watch precision and recall move. This is the same shape as a self-improving agentic loop — act (extract), observe (score), learn (tune), repeat — and it is the mechanism by which the pipeline adapts to new domains without retraining.

The gold set need not be large. Two representative documents, hand-labeled with expected entities and

relations, are enough to establish a baseline and detect regressions. The scorer compares extracted entities against gold names (using an alias map to normalize surface forms), computing precision (fraction of extracted entities that are correct), recall (fraction of gold entities that were extracted), and F1 (the harmonic mean). Relation scoring matches on (source, target) pairs, ignoring predicate wording, giving an upper bound on structural accuracy.

The most common tuning move is adjusting the extraction prompt's selectivity. "Extract only entities central to the document" favors precision; "extract all mentioned entities" favors recall. The right setting depends on the downstream use case: a fact-checking evaluator needs high precision (wrong entities spawn wrong relations), while a research agent needs higher recall (missing entities mean missing connections). The evaluation harness makes this trade-off explicit, measurable, and reversible.

X. LIMITATIONS AND OPEN PROBLEMS

Four limitations are worth stating plainly. First, extraction quality depends on prompt engineering, and the optimal prompt varies by domain — what works on news articles may miss critical distinctions in legal contracts or biomedical literature. Second, entity resolution at scale requires blocking heuristics (grouping candidates by cheap signals before expensive LLM arbitration), and blocking is itself domain-dependent. Third, the graph is only as good as its corpus — biased or incomplete documents produce a biased or incomplete world model. Fourth, temporal reasoning is limited: the current schema lacks edge-level timestamps, so the graph represents what is true but not when it became true or ceased to be true.

Three open problems follow. Temporal graphs — adding time ranges to edges — would enable questions like "who held this role in Q3 2024?" Confidence scoring — weighting edges by cross-document corroboration — would let evaluators distinguish well-supported facts from single-source claims. And federated graphs — multiple teams maintaining domain-specific subgraphs linked by a meta-graph — would enable cross-team reasoning without schema merging. Each extends the same core architecture.

XVI. CONCLUSION

Graph engineering is the discipline that gives multi-agent systems the memory their context windows cannot provide. The three problems RAG cannot solve — multi-hop reasoning, cross-agent state sharing, and provenance-carrying fact-checking — are exactly the problems a knowledge graph addresses. Anthropic's building blocks — structured outputs for schema-constrained extraction, MCP for agent-graph connectivity, and the two-model strategy for cost-optimized processing — make the pipeline practical without external NLP infrastructure.

The integration with agent patterns is specific and actionable: the graph is the retrieval source for the augmented LLM, the gate signal for prompt chaining, the shared memory for orchestrator-workers, and the grounding layer for evaluator-optimizer loops. The evaluation feedback loop — change prompt, rerun scorer, watch F1 — is the mechanism by which the pipeline improves itself.

The central argument restated: as generation becomes cheap and agents multiply, the bottleneck shifts from producing answers to knowing which facts to trust. The knowledge graph is the infrastructure that

makes trust traceable — every edge carries its source, every claim can be checked, and every answer cites the path that produced it. Graph engineering is how you build the memory that agents deserve.

XII. PRACTICAL ARCHITECTURE: THE GRAPH-GROUNDED AGENT

A graph-grounded agent differs from a RAG-augmented agent in three mechanical ways that compound into a qualitative difference in capability. First, the retrieval step traverses typed edges rather than computing vector similarity — the agent asks "what entities are connected to X within 2 hops" rather than "what chunks are similar to my question." Second, the answer is constrained to cite specific edges with provenance rather than quoting text chunks — traceability is structural, not stylistic. Third, the memory persists across sessions without reingestion — the graph is durable, not ephemeral.

The architecture looks like this. A user submits a question. The agent identifies seed entities (either by name-matching or by a fast extraction call). It serializes the k-hop subgraph around those seeds as triples. It feeds the question plus the serialized triples to Sonnet with the instruction "answer using only the graph below; cite specific edges." Sonnet produces a grounded answer. If the answer is insufficient — the subgraph is too small or the question requires entities not in the graph — the agent can expand to k+1 hops or trigger an extraction run on additional documents before retrying.

The key architectural choice is subgraph size. At k=1 (direct neighbors only), the subgraph is small and focused but may miss indirect connections. At k=2 (neighbors of neighbors), the subgraph captures the

two-hop chains that make the graph valuable for multi-hop reasoning. At $k=3$ and beyond, the subgraph may exceed the context window and needs filtering. For most enterprise use cases, $k=2$ with a cap of 200 triples is the sweet spot — large enough for multi-hop reasoning, small enough to fit in a single Sonnet call.

A. The Grounded Evaluator

The most powerful application of graph-grounding is in the evaluator role. Consider an evaluator that receives a draft report from a generator agent. Without a graph, the evaluator reads the report and produces subjective feedback — "paragraph 3 seems unsupported," "the timeline in section 2 doesn't look right." With a graph, the evaluator can systematically check every factual claim: for each entity mentioned in the report, query the graph for its profile and compare. For each relationship asserted, query the graph for the corresponding edge. For each date or number, check the time range in the entity profile. The feedback becomes a structured list of verified and unverified claims, each with the specific graph evidence that supports or contradicts it.

This transforms the evaluator-optimizer loop from a conversation (two agents discussing quality) into an audit (one agent checking claims against a fact store). The distinction matters because conversations can drift, hallucinate, and agree on errors — but an audit against a provenance-carrying graph produces verdicts that are deterministic given the graph state. If the graph says Armstrong commanded Apollo 11 and the report says Armstrong commanded Gemini 12, the evaluator's feedback is not "this seems wrong" but "edge (Armstrong, commanded, Apollo 11) exists

from source X; no edge (Armstrong, commanded, Gemini 12) exists."

B. The Overnight Accumulator

A self-improving loop that runs overnight needs memory that survives context flushes. The graph serves this role naturally. Each overnight run processes new documents (arrived since the last run), extracts entities and relations, resolves them against the existing canonical set, and adds new edges. A state file records which documents have been processed. In the morning, the graph contains everything from last night — entities, relations, provenance, profiles — and any agent can query it without re-reading the source documents. The human reviews a sample of newly added entities, checking provenance and resolution decisions, and flags errors for correction. This is the graph-engineering equivalent of the loop-engineering practice of "read a sample, always."

XIII. SECURITY AND TRUST BOUNDARIES

A knowledge graph used by autonomous agents introduces security considerations that differ from a traditional database. Three boundaries matter.

Read vs. write access. Not every agent should be able to modify the graph. An evaluator agent that reads the graph to check facts should not be able to write to it — otherwise a compromised evaluator could alter the facts it is supposed to check. The MCP pattern supports this naturally: expose a read-only graph tool to evaluators and a read-write tool only to extraction agents with validated provenance.

Provenance integrity. Every edge must trace to a source document, and that provenance must be immutable. If an agent can alter provenance, it can

launder false claims through the graph. The minimal defense is append-only provenance: new edges are added, old edges are superseded but never deleted, and provenance records are write-once.

Query scope. In a multi-tenant system, agents should see only the subgraph relevant to their authorization level. A graph containing both public product data and internal financial data should not expose financial edges to a customer-facing agent. MCP connectors can enforce scope at query time, filtering results by an access-control tag on each edge — but the tag must be set at extraction time, not retroactively applied.

XIV. WORKED EXAMPLE: INCIDENT RESPONSE

To make the architecture concrete, consider a team using a graph-grounded agent system for incident response. The corpus includes runbooks, past incident reports, architecture documents, and vendor contracts.

Extraction (one-time + incremental): Haiku processes each document, extracting entities (services, teams, vendors, incidents, SLAs) and relations (*depends_on*, *owns*, *violated*, *resolved_by*). New documents are processed incrementally as they arrive.

Resolution: Sonnet clusters "Auth Service" / "authentication-svc" / "auth_service" into one canonical node. It merges "Jane Smith" / "J. Smith" / "jsmith@company.com" into one person node using descriptions as context.

Assembly: The graph shows which services depend on which, which teams own which services, which incidents affected which services, and which vendors provide which components. Hub nodes (the

services with the most connections) are summarized with profiles including SLA data and incident history.

Incident triggers: An alert fires for the payment service. The on-call agent queries the graph: "show me everything connected to payment-service within 2 hops." The serialized subgraph reveals: payment-service *depends_on* auth-service, auth-service *depends_on* vendor-X's API, vendor-X had an outage 3 weeks ago (from a past incident report), and the SLA with vendor-X requires 99.9% uptime. The agent's initial assessment: "possible vendor-X dependency issue; check vendor-X status page. Previous incident in the graph: INC-2024-0847, resolved by switching to fallback auth endpoint."

Without the graph, the on-call agent would search Slack, scan runbooks, and try to remember past incidents. With the graph, the multi-hop chain (payment-service → *depends_on* → auth-service → *depends_on* → vendor-X → *had_outage* → INC-2024-0847) is discovered in milliseconds, and every claim traces to a source document.

XV. THE GRAPH ENGINEERING STACK

By analogy with the four-layer stack from loop engineering (prompt → context → harness → loop), graph engineering adds a fifth layer that sits alongside the loop:

TABLE III
THE FIVE-LAYER STACK WITH GRAPH ENGINEERING

Layer	What it minds	Core question
<i>Prompt</i>	What to tell the model	How do I phrase this?
<i>Context</i>	What goes in the window	What to load, summarize, clear?
<i>Harness</i>	Arming a single run	Which tools, what's "done"?
<i>Loop</i>	Scheduling the harness	

Layer	What it minds	Core question
		How to make it run itself?
Graph	Shared memory across agents and sessions	What do agents know together?

The graph layer is not above the loop — it is alongside it, serving the loop as its persistent memory and serving multi-agent systems as their shared state. A loop without a graph forgets between runs. A graph without a loop is a static database. Together, the loop builds and refines the graph over time, and the graph gives the loop a world model that accumulates rather than resets.

This framing clarifies when to invest in graph engineering. If your system is a single agent answering questions in a single session, the prompt and context layers are sufficient. If it runs on a schedule, you need the loop layer. If it runs multiple agents that must coordinate, or if it runs overnight and must remember what it found yesterday, you need the graph layer. Each layer earns its complexity by solving a problem the layers below it cannot.

XVI. ANTI-PATTERNS IN GRAPH ENGINEERING

Seven anti-patterns recur in teams adopting knowledge graphs for agentic systems. *The phantom graph*: an elaborate ontology that no agent queries — designed before needs were understood. Fix: build the schema from real queries, start with three entity types. *The monolith extract*: one prompt tries to find every entity type, splitting attention too thin. Fix: route documents by type, then run domain-specific extraction. *The resolution leak*: duplicated entities fragment paths silently. Fix: monitor connected components. *The stale graph*: built once, never updated. Fix: incremental extraction on new documents. *The ungrounded*

evaluator: has graph access but never queries it. Fix: require fact-checking against graph edges in the evaluator’s prompt. *The context dump*: serializing the entire graph rather than a targeted subgraph. Fix: seed + k-hop neighborhood only. *The write-only graph*: agents extract but never query. Fix: wire graph-read into every decision-making workflow.

XVII. DECISION FRAMEWORK

TABLE III
WHEN TO USE A KNOWLEDGE GRAPH VS. ALTERNATIVES

Use-case signal	Right tool	Why
Single-document QA	Direct context	Answer is in one passage
Multi-doc, single-hop	RAG	Spans docs but no chaining
Multi-doc, multi-hop	<i>Knowledge graph</i>	Chaining requires entity linking
Multi-agent co-ordination	<i>Knowledge graph</i>	Shared state outside windows
Fact-checking	<i>Knowledge graph</i>	Needs provenance-carrying facts
Cross-session persistence	<i>Knowledge graph</i>	Memory survives flushes
Simple classification	Single agent	No cross-doc reasoning

The decision rule: if your agents need to *chain* facts, *share* structured state, or *ground* judgments in traceable evidence — the knowledge graph is the right infrastructure. Otherwise, simpler tools suffice.

XVIII. PRODUCTION CHECKLIST

TABLE IV
PRODUCTION READINESS CHECKLIST

Element	Ask yourself	Risk if missing
<i>Gold set</i>	Labeled entities for ≥ 2 documents?	Blind prompt changes
<i>Alias map</i>	Scorer covers all canonical forms?	Distorted metrics

Element	Ask yourself	Risk if missing
<i>Schema version</i>	Extraction schema versioned?	Incompatible entities
<i>Extraction cap</i>	Per-run document limit?	Unbounded cost
<i>Resolution fallback</i>	Unmatched names → single cluster?	Silent entity loss
<i>Provenance</i>	Every edge → source doc + timestamp?	Evaluator can't verify
<i>Incremental update</i>	New docs without full rebuild?	Rebuild cost scales with corpus
<i>Connectivity monitor</i>	Component count checked?	Silent fragmentation
<i>Read/write separation</i>	Evaluators read-only?	Compromised evaluator alters facts
<i>Human sample</i>	Random node reviewed daily?	Graph outgrows understanding

A pipeline with all ten is production-ready. A pipeline missing any has a specific, nameable risk that will surface eventually.

Term	Meaning
<i>Graph engineering</i>	The discipline of designing the shared memory layer for multi-agent systems.
<i>Knowledge graph</i>	Entities as nodes, typed relationships as edges, with provenance on every triple.
<i>Structured outputs</i>	API feature constraining Claude's response to validate against a Pydantic schema.
<i>Entity resolution</i>	Merging different surface forms of the same real-world entity into one canonical node.
<i>Multi-hop reasoning</i>	Answering a question by chaining facts across multiple graph edges.
<i>MCP</i>	Model Context Protocol — the open standard connecting agents to external tools and data.
<i>Provenance</i>	The source document and extraction context from which a triple was derived.
<i>Grounded answer</i>	An answer constrained to extracted graph edges, with citations, not drawn from pretraining.
<i>Evaluation harness</i>	The feedback loop: change prompt → rerun scorer → watch precision/recall move.
<i>Two-model strategy</i>	Haiku for high-volume extraction, Sonnet for resolution and reasoning — routing by cost.

ACKNOWLEDGMENT & SOURCES

This document is an independent synthesis for study, not affiliated with or endorsed by Anthropic. The knowledge-graph pipeline and evaluation methodology are from "**Knowledge Graph Construction with Claude**," Anthropic's public claude-cookbooks repository (Mar. 2026). Agent patterns and design principles are from **E. Schluntz & B. Zhang**, "Building Effective Agents," Anthropic Engineering, Dec. 2024, and from "**Building Effective AI Agents: Architecture Patterns and Implementation Frameworks**," Anthropic, 2026. The Managed Agents architecture is from **L. Martin, G. Cemaj & M. Cohen**, "Scaling Managed Agents," Anthropic Engineering, Apr. 2026. The MCP specification and Knowledge Graph Memory Server are from Anthropic's open-source MCP repositories. Structured outputs documentation is from the Claude API platform docs. All diagrams are original. Product capabilities cited reflect source material at time of writing and may have changed.

APPENDIX: GLOSSARY

TABLE II
TERMS USED IN THIS NOTE

This document is an independent synthesis assembled for study. It is not a publication of, and is not affiliated with or endorsed by, Anthropic. The knowledge-graph pipeline is from Anthropic's public claude-cookbooks repository. Agent architecture patterns are from Anthropic's published engineering writing. Full attribution in Sources. Diagrams are original.